

第十二章：用户态

一、核心概念

1.1 用户态与内核态的边界

用户程序运行在最低特权级（RISC-V U-mode），不能直接：

- 执行特权指令（修改 CSR、关中断）
- 访问内核内存（PTE_U=0 的页）
- 操作硬件设备

所有内核服务必须通过**系统调用**（ecall）请求。

1.2 C 运行时（CRT, C Runtime）

C 程序的 `main()` 不是真正的入口点。在 `main` 之前需要：

- 设置栈指针
- 初始化 BSS 段
- 处理命令行参数（`argc, argv`）
- 调用全局构造函数（C++）
- 调用 `main(argc, argv)`
- `main` 返回后调用 `exit()`

这些工作由 CRT（`crt0.o` / `crt1.o`）完成。

1.3 用户态内存分配

brk/sbrk：调整进程的"程序断点"（堆顶），是最原始的堆内存管理接口。

- `sbrk(n)`：将堆顶向上移动 `n` 字节，返回旧堆顶
- `brk(addr)`：将堆顶设为 `addr`

malloc 实现：在 `brk/sbrk` 之上构建空闲链表管理器。经典算法：

- **K&R malloc**：first-fit，空闲链表，合并相邻块
- **dlmalloc**：分级 free list，更好的碎片管理
- **glibc malloc (ptmalloc2)**：per-thread arena，减少锁竞争
- **jemalloc/tcmalloc**：现代高性能分配器，thread cache

1.4 动态链接 vs 静态链接

	静态链接	动态链接
时机	编译时	运行时
文件大小	大（包含所有库代码）	小（共享库）
内存	每个进程一份	多进程共享库代码页

静态链接		动态链接
更新	需重新编译	替换 .so 即可
实现	简单	需要 ld.so、GOT/PLT

ELF 中的 PT_INTERP：动态链接的 ELF 有一个 **PT_INTERP** segment，指定动态链接器路径（如 `/lib/ld-musl-riscv64.so.1`）。内核加载 ELF 时先加载 interpreter，由它完成符号解析和重定位。

1.5 ELF 文件格式关键概念

概念	说明
ELF Header	魔数、架构、入口地址、Program Header 偏移
Program Header	描述内存段（LOAD、INTERP、DYNAMIC 等）
Section Header	描述链接时的节（.text, .data, .bss, .got, .plt）
GOT (Global Offset Table)	存放全局变量/函数的运行时地址
PLT (Procedure Linkage Table)	函数调用的延迟绑定跳板

二、基本推论

- 推论 1**：用户态的 **libc** 是系统调用的封装层。 `printf()` → `write()` → `ecall`。不同的 libc（glibc, musl, uclibc）提供相同的 POSIX API 但实现不同。
- 推论 2**：**malloc** 是用户态的"内存管理子系统"。brk/mmap 是向内核要内存的接口（页级粒度），malloc 在用户态做细粒度管理（字节级），和内核的 slab 思想类似。
- 推论 3**：**动态链接是内存效率和部署灵活性的基础**。100 个进程共享一份 libc.so 的代码页，比 100 份静态链接的 libc 代码节省大量内存。

三、Linux / 通用实现

3.1 Linux 的 exec 加载 ELF

```
do_execve() → load_elf_binary()
1. 读 ELF header + Program Headers
2. 检查 PT_INTERP → 如果有，加载动态链接器
3. 遍历 PT_LOAD → mmap 到用户地址空间
4. 设置 auxv (辅助向量: AT_ENTRY, AT_PHDR, AT_BASE 等)
5. 设置栈: argv + envp + auxv
6. 设 entry point:
   - 静态链接 → e_entry
   - 动态链接 → ld.so 的 entry (ld.so 再跳到程序 entry)
```

3.2 glibc 的 malloc

小请求 (< 128KB): 从 arena 的 bins 中分配
 fastbin: 16-80 bytes, 单链表, LIFO
 smallbin: 16-512 bytes, 双链表
 largebin: > 512 bytes, 有序双链表
 unsortedbin: 回收暂存区

大请求 (>= 128KB): 直接 mmap, 独立映射, 释放时 munmap

3.3 musl libc

比赛测试程序用 musl 编译。musl 特点:

- 比 glibc 小很多 (~1MB vs ~10MB)
- 静态链接友好
- 符合 POSIX 标准, 接口兼容
- 动态链接器: `ld-musl-riscv64.so.1`

四、RUOS 的实现

核心文件

- `user/initcode.c` — 第一个用户进程
- `user/ulib.c` — syscall 封装库
- `user/umalloc.c` — K&R malloc
- `user/entry.S` — 用户程序入口
- `user/coro.c` — 无栈协程
- `user/sh.c` — 简易 shell

4.1 第一个用户进程

编译链:

```
initcode.c + entry.S + ulib.o + ... → user/initcode (ELF)
→ objcopy → initcode.bin (纯二进制)
→ 内核 initcode.S: .incbin "user/initcode"
→ userinit(): 复制到第一个进程 → 设 epc → 运行
```

initcode 职责: mount 根文件系统 → 遍历测试目录 → fork+exec 测试程序 → 打印汇总 → 关机。

4.2 syscall 封装

```
static inline long syscall(long num, long a0, ..., long a5) {
    register long _a0 asm("a0") = a0;
    // ... a1-a5
    register long _a7 asm("a7") = num;
    asm volatile("ecall" : "+r"(_a0) : ... : "memory");
    return _a0;
}
```

```

}

// Linux RISC-V ABI 适配:
int fork(void) { return syscall(SYS_clone, SIGCHLD, 0, ...); }
int open(...) { return syscall(SYS_openat, AT_FDCWD, ...); }
int pipe(int *) { return syscall(SYS_pipe2, ..., 0); }

```

4.3 K&R malloc

```

typedef union header {
    struct { union header *ptr; uint size; } s;
    long x; // 对齐
} Header;
// 空闲链表循环链表, first-fit, sbrk 扩展

```

简单可靠，适合教学 OS。

4.4 无栈协程

利用 `__LINE__` 做 switch/case 状态机：

```

#define CORO_BEGIN(ctx)  switch(ctx->state) { case 0:
#define CORO_YIELD(ctx)  do { ctx->state=__LINE__; return; case __LINE__;;
} while(0)
#define CORO_END(ctx)    } ctx->state = -1;

```

用于网络测试中"同时"运行 server 和 client。

4.5 用户地址空间布局

```

0x00001000 .text / .rodata / .data / .bss
            heap ← sbrk 向高地址增长
            ...
            mmap 区域
            ...
            stack (USERSTACK_PAGES=8, 32KB) ← 向低地址增长
            [guard page]
TRAPFRAME  trapframe (内核映射)
TRAMPOLINE trampoline (共享跳板页)

```

五、八股 / 面试高频问题

Q: ELF 文件中 Section 和 Segment 的区别? A: Section 是链接视图 (.text, .data, .bss, .rodata)，Segment 是执行视图 (PT_LOAD, PT_INTERP)。链接器操作 Section，加载器操作 Segment。一个 Segment 可包含多

个 Section。

- Q: 动态链接的过程? GOT/PLT 是什么?** A: 程序首次调用外部函数 → PLT 跳板 → 检查 GOT 条目 → 首次是 ld.so 的 resolver → 解析符号地址 → 写入 GOT → 后续直接跳到 GOT 中的地址（延迟绑定）。
- Q: malloc 的内存从哪来?** A: 小请求用 brk/sbrk 扩展堆；大请求用 mmap 直接映射匿名页。free 不一定归还给 OS——只是放回 malloc 的空闲链表，供后续 malloc 复用。
- Q: 栈溢出会发生什么?** A: 如果有 guard page，触发 Page Fault → 内核杀死进程（SIGSEGV）。没有 guard page，可能默默覆盖其他数据（栈溢出攻击的基础）。
- Q: 用户态和内核态的寻址空间如何划分?** A: RISC-V Sv39: 低 256GB 是用户空间，高 256GB 是内核空间。x86-64: 低 128TB 用户，高 128TB 内核（典型配置）。内核页表映射全部物理内存，用户页表只映射该进程的地址空间。

六、项目贡献亮点

- 实现了完整的用户态 syscall 封装库，适配 Linux RISC-V ABI (clone 代替 fork, openat 代替 open)
- 实现了动态链接 ELF 的 PT_INTERP 解析和加载（适配 musl 编译的测试程序）
- 用户态 K&R malloc 基于 sbrk，与内核的 buddy+slab 形成两层内存管理体系

七、设计取舍总结

决策	RUOS 选择	Linux 做法	权衡
libc	自写 ulib.c	glibc/musl	最小化，够用
malloc	K&R first-fit	ptmalloc2/jemalloc	简单但碎片多
动态链接	基本 PT_INTERP 支持	完整 ld.so	满足 musl 测试
shell	简易 sh	bash/zsh	基本管道/重定向
程序入口	纯二进制嵌入	ELF + initramfs	简单直接